SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE:CSA5110 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 20%

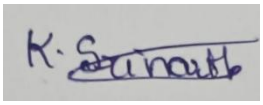
QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
CASE STUDY

Code of Conduct:

I K.Srinath(192524220)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

1. Weak Password Hashing in Web Application
A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.
Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

---

2. Digital Signature Compromise in Financial System
A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.
Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

---

3. Kerberos Authentication Failure in Distributed System
A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.
Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

---

4. Certificate Authority Breach in PKI System
A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.
Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

---

5. Integrity Verification in Cloud Data Storage
A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.
Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. Weak Password Hashing in Web Application

Introduction

Password security is a fundamental part of web application security. Web applications need to store passwords in a form that prevents an attacker who obtains the database from immediately learning the original passwords. For this reason, passwords should be processed using a dedicated password-hashing algorithm before being stored.

In the given scenario, the web application stores passwords using **unsalted SHA-1 hashes**. This creates a major vulnerability because SHA-1 is a fast general-purpose hash function and the absence of salt makes the stored hashes vulnerable to precomputed-table, dictionary, and brute-force attacks. After a database breach, attackers can therefore recover many weak passwords.

---

1. Password Hashing

Hashing is a one-way process that converts input data into a fixed-length value.

The general process is:

Password → Hash Function → Hash Value

Unlike encryption, hashing is not intended to be reversed to obtain the original password.

During login, the application processes the entered password using the same password-hashing procedure and compares the result with the stored password hash.

A secure password-storage system should therefore never store:

- Plaintext passwords
- Reversible encrypted passwords as a substitute for password hashing
- Fast unsalted hashes such as SHA-1

---

2. Why SHA-1 is Inappropriate

SHA-1 was designed as a general-purpose cryptographic hash function, not as a password-storage algorithm.

One of its major disadvantages for password storage is its speed. Password hashing should be deliberately expensive so that legitimate authentication remains practical while large-scale password guessing becomes costly for attackers.

If an attacker steals a database containing SHA-1 hashes, they can generate a very large number of password guesses and compare the resulting hashes with the stolen values.

Therefore:

Fast hashing is an advantage for normal hashing applications but a disadvantage for password storage.

---

3. Lack of Salt

A **salt** is a random value added to a password before hashing.

Without salt:

Password → SHA-1 → Hash

If two users select the same password, the same hash will be stored.

For example:

User A → welcome123 → Hash X

User B → welcome123 → Hash X

An attacker can immediately recognize that both accounts use the same password.

With unique salts:
User A → welcome123 + Salt A → Hash X
User B → welcome123 + Salt B → Hash Y
The hashes become different.

---

4. Importance of Unique Salts
A unique salt provides several security benefits:
a. Prevents identical hashes
Users with the same password will have different stored hashes.
b. Reduces effectiveness of precomputed tables
An attacker cannot simply compare a stolen hash with a universal precomputed table.
c. Makes mass cracking more expensive
The attacker has to perform password-guessing work separately for each salt.
d. Protects against rainbow-table attacks
Rainbow tables become far less practical when every password has a unique salt.

---

5. Precomputed Table Attack
In a precomputed-table attack, an attacker prepares a large collection of passwords and their corresponding hashes.
For example:

Password	Hash
123456	Hash A
password	Hash B
admin123	Hash C
welcome123	Hash D

If the stolen database contains one of these hashes, the attacker can identify the corresponding password quickly.
Because the application does not use a salt, the same precomputed value can potentially be useful against many users.

---

6. Brute-Force and Dictionary Attacks
Brute-Force Attack
The attacker tries many combinations until the generated hash matches the stolen hash.
For example:
123456
123457
123458
...
password1
password12
password123
Dictionary Attack
The attacker uses a list of commonly used passwords.
Examples include:
password

admin
qwerty
welcome
123456
letmein
The use of a fast hashing algorithm makes these attacks more efficient.

---

7. Impact of the Vulnerability

If attackers crack the passwords, they may be able to:

- Take over user accounts
- Access personal information
- Steal sensitive data
- Perform unauthorized actions
- Use stolen credentials on other websites
- Conduct identity theft
- Cause financial losses

Password reuse makes the situation even more dangerous because a password obtained from one website may work on other services.

Conclusion

The use of **unsalted SHA-1** is a serious weakness because SHA-1 is fast and identical passwords generate identical hashes. The lack of salt makes precomputed tables and large-scale password cracking much more effective. The application should migrate to **Argon2id with a unique random salt for every password**, supported by MFA, rate limiting, secure password resets, HTTPS, and proper database access controls. These measures significantly improve resistance against password cracking and account takeover.

2. Digital Signature Compromise in Financial System

Introduction

Digital signatures are an important part of security in financial systems. They are used to verify the authenticity and integrity of electronic transactions and provide evidence that a particular cryptographic key was used to authorize a transaction.

A digital-signature system uses a **private key for signing** and a corresponding **public key for verification**. The private key must remain confidential. If an attacker obtains the private key, they may be able to generate signatures that pass normal verification and use them to authorize fraudulent transactions.

---

1. Working of Digital Signatures

The basic signing process is:

Transaction
↓
Hash Function
↓
Message Digest
↓
Private Key
↓
Digital Signature

The financial institution can verify the signature using the corresponding public key.

Transaction + Signature
↓
Public Key
↓
Verification
↓
Valid / Invalid

If the transaction is changed after signing, the signature verification should fail.

---

2. Importance of the Private Key

The private key is the most sensitive component in a digital-signature system.

It should:

- Remain confidential
- Be accessible only to authorized users or cryptographic services
- Be securely stored
- Be protected against malware
- Be backed up using secure procedures where appropriate
- Be revoked immediately if compromised

If the private key is stolen, the attacker may be able to create fraudulent signatures.

---

3. Effect on Authentication

Authentication establishes the identity of the party associated with a transaction.

Normally:

User
↓
Private Key
↓
Signature
↓

Verification
↓
User Authenticated
After private-key compromise:
Attacker
↓
Stolen Private Key
↓
Signature
↓
Verification
↓
Appears to be legitimate
Therefore, the financial institution may incorrectly treat an attacker as the legitimate user.
This results in a serious **authentication failure**.

---

4. Effect on Non-Repudiation
Non-repudiation provides evidence that a particular private key was used to sign a transaction.
For example:
Transaction
↓
Private Key
↓
Digital Signature
↓
Evidence of Signing
However, private-key compromise creates uncertainty.
If an unauthorized transaction was signed using a compromised key, the legitimate user may deny authorizing it and report that the key was stolen.
Therefore, the organization needs to investigate:

- When the key was compromised
- When the transaction occurred
- Whether the key had already been reported as compromised
- Whether the certificate was valid at that time
- Whether additional authentication was used

Thus, key compromise weakens the practical strength of non-repudiation for affected transactions.

---

5. Effect on Integrity
Digital signatures normally help detect unauthorized modification of transaction data.
For example:
Original:
Transfer ₹10,000
If an attacker changes it to:
Transfer ₹1,00,000
the original signature should no longer verify.
However, if the attacker has the private key, they may potentially create a new signature for the modified transaction.
Therefore, protecting the private key is essential for maintaining the integrity guarantees of the signature system.

---

6. Possible Consequences
A compromised private key can potentially result in:

- Unauthorized money transfers
- Fraudulent transaction approvals
- Impersonation of users
- Unauthorized financial contracts
- Customer losses
- Legal disputes
- Loss of customer trust
- Damage to the institution's reputation

The exact impact depends on the privileges assigned to the compromised key.

---

7. Causes of Private-Key Compromise

Private keys can be exposed through:

Malware

Malicious software can steal credentials or access key material.

Phishing

Attackers can trick users into revealing authentication information or installing malicious software.

Insecure Storage

A private key stored as an unprotected file can be copied by an attacker.

Lost or Stolen Devices

A laptop, smart card, or other signing device may be physically stolen.

Insider Threats

Employees or administrators with excessive privileges may misuse access.

Weak Key Management

Poor key generation, storage, rotation, or revocation practices can increase risk.

---

8. Mitigation Strategies

1. Hardware Security Modules

High-value financial keys should be protected using **Hardware Security Modules (HSMs)**.

An HSM securely stores keys and performs cryptographic operations without exposing the private key to normal applications.

Application
↓
HSM
↓
Digital Signature

This significantly improves private-key protection.

Conclusion

A digital-signature system depends heavily on the security of its private keys. If an attacker gains control of a user's private key, they may be able to create valid-looking signatures and perform unauthorized transactions. This can compromise **authentication, integrity, authorization, and the practical assurance of non-repudiation**.

Financial institutions should therefore use **HSMs, MFA, secure key storage, key rotation, certificate revocation, transaction monitoring, transaction limits, separation of duties, and secure audit logs**. A strong incident-response process should also be used to immediately revoke compromised keys and investigate affected transactions. By combining these measures, the institution can significantly reduce the risk and financial impact of private-key compromise.